SPI BASED EEPROM CONTROLLER

Design Module

```
None
module spi_eeprom_controller (
    input clk,
    input rst,
    input start,
    input wr,
    input [7:0] address,
    input [7:0] write_data,
    input miso,

    output reg mosi,
    output reg sclk,
    output reg cs,
    output reg [7:0] read_data,
    output reg done
);

    parameter IDLE = 2'b00,
              CMD  = 2'b01,
              ADDR = 2'b10,
              DATA = 2'b11;

    reg [1:0] state;
    reg [7:0] shift_reg;
    reg [7:0] addr_reg;
    reg [7:0] data_reg;
    reg [2:0] bit_count;
    reg [1:0] phase;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            state     <= IDLE;
            sclk      <= 0;
            cs        <= 1;
            mosi      <= 0;
            done      <= 0;
            bit_count <= 0;
            phase     <= 0;
            read_data <= 0;
```

```
    end else begin
        case (state)

            IDLE: begin
                cs   <= 1;
                sclk <= 0;
                done <= 0;

                if (start) begin
                    cs        <= 0;
                    addr_reg  <= address;
                    data_reg  <= write_data;
                    shift_reg <= wr ? 8'h02 : 8'h03;
                    bit_count <= 0;
                    phase     <= 0;
                    mosi      <= wr ? 0 : 0;
                    state     <= CMD;
                end
            end

            CMD: begin
                if (phase == 0) begin
                    mosi  <= shift_reg[7];
                    sclk  <= 0;
                    phase <= 1;
                end else if (phase == 1) begin
                    sclk  <= 1;
                    shift_reg <= {shift_reg[6:0], 1'b0};
                    phase <= 2;
                end else begin
                    sclk  <= 0;
                    phase <= 0;

                    if (bit_count == 7) begin
                        bit_count <= 0;
                        shift_reg <= addr_reg;
                        state <= ADDR;
                    end else begin
                        bit_count <= bit_count + 1;
                    end
                end
            end

            ADDR: begin
                if (phase == 0) begin
```

```
            mosi  <= shift_reg[7];
            phase <= 1;
        end else if (phase == 1) begin
            sclk  <= 1;
            shift_reg <= {shift_reg[6:0], 1'b0};
            phase <= 2;
        end else begin
            sclk  <= 0;
            phase <= 0;

            if (bit_count == 7) begin
                bit_count <= 0;

                if (wr) begin
                    shift_reg <= data_reg;
                    state <= DATA;
                end else begin
                    state <= DATA;
                    shift_reg <= 0;
                end
            end else begin
                bit_count <= bit_count + 1;
            end
        end
    end

    DATA: begin
        if (phase == 0) begin
            if (wr)
                mosi <= shift_reg[7];
            phase <= 1;
        end else if (phase == 1) begin
            sclk <= 1;

            if (!wr)
                shift_reg <= {shift_reg[6:0], miso};
            else
                shift_reg <= {shift_reg[6:0], 1'b0};

            phase <= 2;
        end else begin
            sclk  <= 0;
            phase <= 0;

            if (bit_count == 7) begin
```

```
                            if (!wr)
                                read_data <= shift_reg;

                            cs    <= 1;
                            done  <= 1;
                            state <= IDLE;
                            bit_count <= 0;
                        end else begin
                            bit_count <= bit_count + 1;
                        end
                    end
                end

            endcase
        end
    end
endmodule
```

Testbench

None

```
`timescale 1ns/1ps

module tb_spi_eeprom_controller;

    reg clk;
    reg rst;
    reg start;
    reg wr;
    reg [7:0] address;
    reg [7:0] write_data;
    reg miso;

    wire mosi;
    wire sclk;
    wire cs;
    wire [7:0] read_data;
    wire done;

    spi_eeprom_controller dut (
        .clk(clk),
        .rst(rst),
```

```
    .start(start),
    .wr(wr),
    .address(address),
    .write_data(write_data),
    .miso(miso),
    .mosi(mosi),
    .sclk(sclk),
    .cs(cs),
    .read_data(read_data),
    .done(done)
);

// Clock generation: 10 ns period
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// Test sequence
initial begin
    rst = 1;
    start = 0;
    wr = 0;
    address = 8'h00;
    write_data = 8'h00;
    miso = 0;

    #20;
    rst = 0;

    // WRITE operation
    #10;
    wr = 1;
    address = 8'h10;
    write_data = 8'hA5;
    start = 1;

    #10;
    start = 0;

    wait(done == 1);
    #10;

    // READ operation
    wr = 0;
```

```
        address = 8'h10;
        miso = 1;
        start = 1;

        #10;
        start = 0;

        wait(done == 1);
        #10;

        $finish;
    end

    initial begin
        $monitor(
            "Time=%0t rst=%b start=%b wr=%b addr=%h wdata=%h cs=%b sclk=%b mosi=%b
miso=%b read_data=%h done=%b",
            $time, rst, start, wr, address, write_data,
            cs, sclk, mosi, miso, read_data, done
        );
    end

endmodule
```

